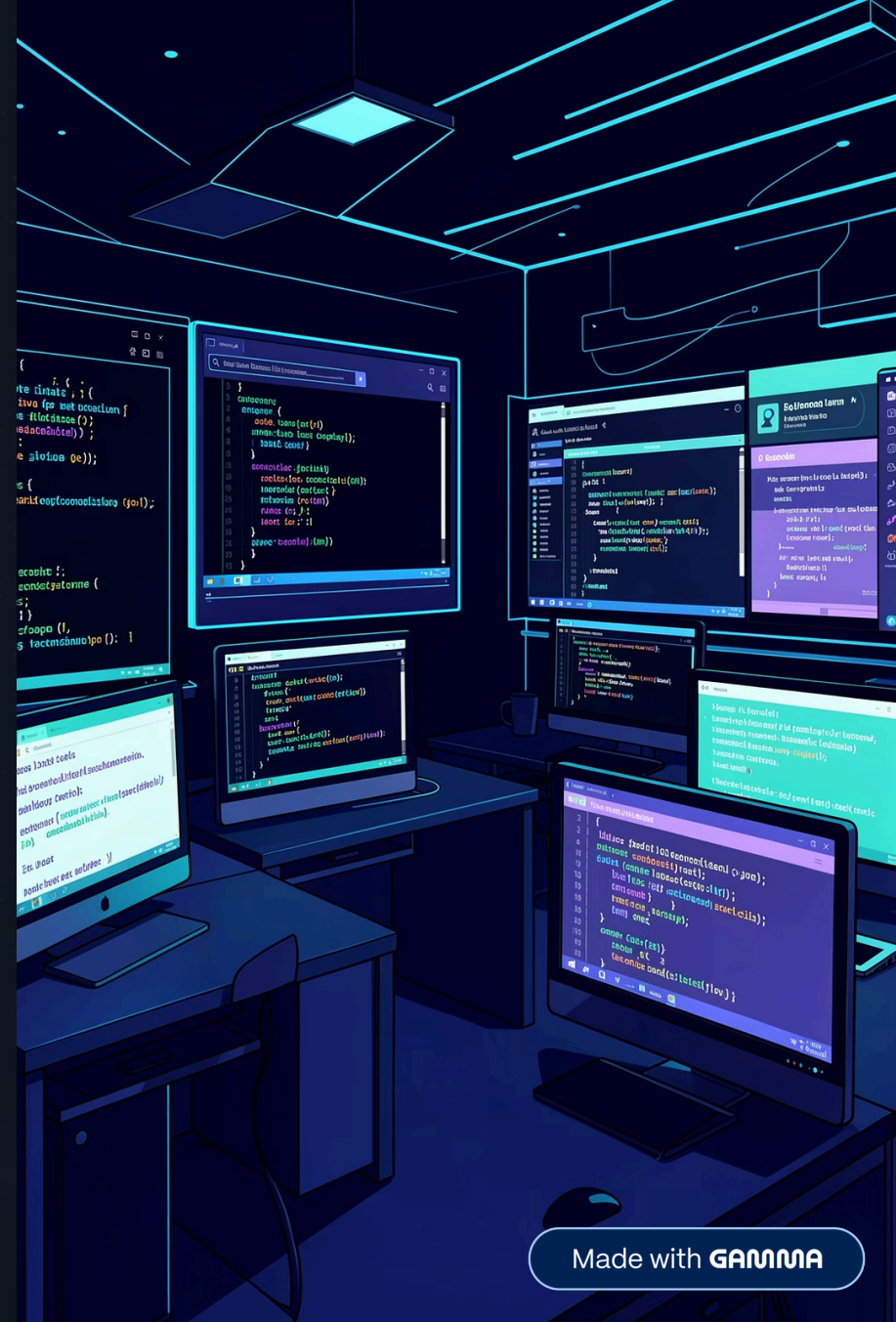
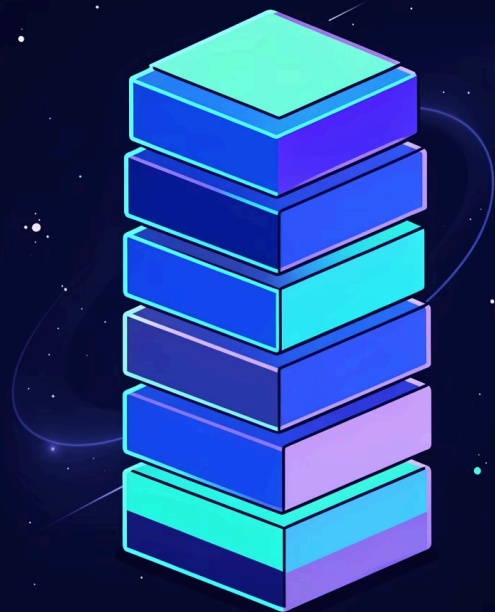


Implementación de Pilas y Colas utilizando Clases en Java

Asignatura: Desarrollo de Software | Estructuras de Datos Fundamentales



Concepto de Pila y Cola en Java



Pila (Stack) - LIFO

Last In, First Out: el último elemento en entrar es el primero en salir. Similar a una pila de platos donde solo accedemos al elemento superior.

Cola (Queue) - FIFO

First In, First Out: el primer elemento en entrar es el primero en salir. Comparable a una fila de personas esperando ser atendidas.

Importancia: Estas estructuras son fundamentales en POO para gestionar datos de forma ordenada y eficiente en aplicaciones reales.

Diseño de una Clase Pila

Atributos Principales

- Array o ArrayList para almacenar elementos
- Variable `tope` para controlar la posición actual
- Constante `CAPACIDAD_MAXIMA` para limitar el tamaño

Métodos Esenciales

- `push(elemento)`: añade un elemento en el tope
- `pop()`: extrae y retorna el elemento del tope
- `peek()`: consulta el tope sin extraerlo

Control de Estado

- `estaVacía()`: verifica si no hay elementos
- `estaLlena()`: verifica si alcanzó la capacidad
- Manejo de excepciones para operaciones inválidas

Implementación Básica de una Pila en Java

Estructura de la Clase

La clase `Pila` encapsula un array interno y gestiona las operaciones mediante métodos públicos que controlan el acceso a los datos.

Uso de arrays: Proporcionan acceso rápido y eficiente para operaciones de inserción y extracción en el tope.

Control de índices: La variable `tope` se incrementa con `push` y se decrementa con `pop`, manteniendo la integridad de la estructura LIFO.

```
public class Pila {  
    private int[] elementos;  
    private int tope;  
    private static final int MAX = 100;  
  
    public Pila() {  
        elementos = new int[MAX];  
        tope = -1;  
    }  
  
    public void push(int elemento) {  
        if (!estaLlena()) {  
            elementos[++tope] = elemento;  
        }  
    }  
  
    public int pop() {  
        if (!estaVacia()) {  
            return elementos[tope--];  
        }  
        return -1;  
    }  
}
```

Diseño de una Clase Cola

Atributos Fundamentales

- Array circular o lista enlazada
- Índice `frente`: posición del primer elemento
- Índice `fin`: posición del último elemento
- Variable `tamaño`: cantidad de elementos actuales

Métodos Principales

- `enqueue(elemento)`: inserta al final de la cola
- `dequeue()`: extrae del frente de la cola
- `verFrente()`: consulta sin extraer

Gestión de Índices

- Control circular: uso de módulo para reutilizar espacios
- Validación de cola llena y vacía
- Mantenimiento del orden FIFO estricto

Implementación Básica de una Cola en Java

```
public class Cola {  
    private int[] elementos;  
    private int frente, fin, tamaño;  
    private static final int CAPACIDAD = 100;  
  
    public Cola() {  
        elementos = new int[CAPACIDAD];  
        frente = 0;  
        fin = -1;  
        tamaño = 0;  
    }  
  
    public void enqueue(int elemento) {  
        if (tamaño < CAPACIDAD) {  
            fin = (fin + 1) % CAPACIDAD;  
            elementos[fin] = elemento;  
            tamaño++;  
        }  
    }  
  
    public int dequeue() {  
        if (!estaVacia()) {  
            int elemento = elementos[frente];  
            frente = (frente + 1) % CAPACIDAD;  
            tamaño--;  
            return elemento;  
        }  
        return -1;  
    }  
}
```

Características Clave

Array circular: Optimiza el uso de memoria reutilizando posiciones liberadas mediante operación módulo.

Gestión FIFO: El índice `frente` siempre apunta al próximo elemento a extraer, mientras `fin` señala dónde insertar.

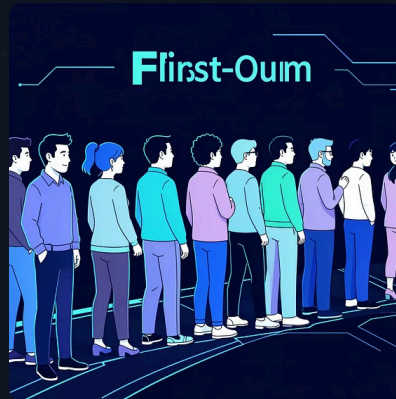
Control de tamaño: Evita desbordamientos y permite verificar el estado de la cola eficientemente.

Comparación entre Pila y Cola



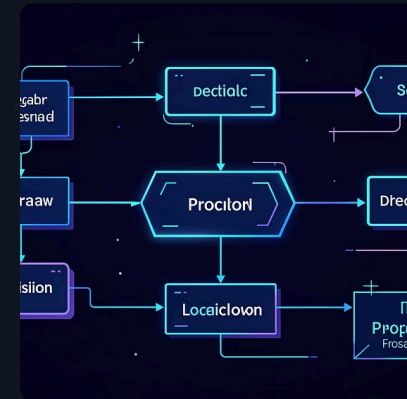
Orden de Acceso

Pila: Acceso restringido al último elemento insertado (LIFO). **Cola:** Acceso al primer elemento insertado (FIFO).



Casos de Uso

Pilas: Deshacer/rehacer, navegación de historial, recursión. **Colas:** Gestión de tareas, impresión, atención de solicitudes.



Impacto en Lógica

La elección entre pila y cola determina el flujo de procesamiento y afecta directamente la eficiencia del algoritmo implementado.

Casos de Aplicación Real



Las pilas son esenciales para gestionar el historial de navegadores web y llamadas a funciones en tiempo de ejecución. Las colas se utilizan en sistemas operativos para planificación de procesos y en aplicaciones cliente-servidor para gestionar solicitudes entrantes de manera ordenada.

Buenas Prácticas y Principios de Diseño

1

Encapsulamiento Riguroso

Declarar atributos como `private` y proporcionar métodos públicos controlados. Esto protege la integridad de la estructura y evita modificaciones directas no autorizadas.

2

Validación de Operaciones

Implementar verificaciones antes de `push/pop` y `enqueue/dequeue`. Lanzar excepciones específicas (`StackOverflowException`, `EmptyQueueException`) para errores identificables.

3

Uso de Genéricos

Implementar `Pila<T>` y `Cola<T>` con tipos genéricos para reutilización con diferentes tipos de datos, mejorando la flexibilidad del código.

4

Documentación y Pruebas

Incluir `JavaDoc` en todos los métodos públicos. Crear casos de prueba unitarios que verifiquen condiciones límite y comportamiento esperado en cada operación.



Conclusiones y Aplicaciones Prácticas



Fundamentos Sólidos

Dominar pilas y colas es esencial para comprender estructuras de datos más complejas y algoritmos avanzados en desarrollo de software profesional.



Implementación Efectiva

La correcta implementación mediante clases Java garantiza código mantenible, reutilizable y eficiente para proyectos de cualquier escala.



Aplicación Profesional

Estas estructuras son utilizadas diariamente en frameworks empresariales, sistemas operativos y aplicaciones de alto rendimiento en la industria del software.